

Intervention-Based Identification of Self-State Representations

A Test of Functional Self-Reference (C-IV)

Taotuner — September 12, 2026

Summary

We test whether a p-adic algebraic system can identify, among two structurally symmetric internal representations whose labels carry no information about their role, which one is causally coupled to the system's own state, and then use the identified representation for control. The driver is chosen at random for each run; the controller is not informed which variable it is and must determine it by intervention.

Three results are reported:

Identification. A parallel-trajectory intervention produces exact separation between the coupled and uncoupled representations. Across 20 randomized driver assignments (5 with driver = A, 15 with driver = B), the coupled representation diverged by 15 and the uncoupled representation by 0 in every run, out of a maximum possible circular divergence of 18 (the distance measure is defined explicitly in §2.1).

Functional use. With the identified representation, the system maintains a target invariant for exactly 499 of 500 control steps in every one of the 20 seeds. With the wrong representation, viability drops to approximately 0.3% — a figure in the same range as the 0–2 viable steps out of 500 (0–0.4%) observed for δ values outside the tolerance set in §4.2, though the wrong-representation condition itself is not run separately in the Appendix code below and should be read as consistent with, rather than independently confirmed by, that sweep.

Content dependence. Corrupting the copy of the identified representation used by the policy — while the internal simulation of the state remains uncorrupted — shifts the closed-loop fixed point by 2δ modulo 13. The observed viability pattern matches the analytically predicted modular tolerance set for all 13 values of δ .

The three operational components specified for C-IV in the HDD-ISA framework — representation of the system's own state, causal use dependent on content, and intervention-based discrimination between coupled and uncoupled representations — are demonstrated within the tested architecture and the assumptions stated in §6. The result should not be interpreted as evidence for phenomenal selfhood or semantic self-awareness, nor does it establish robustness, spontaneous probe generation, or scalability.

1. Context

The Zeta system [1] is a p-adic algebraic structure over $\mathbb{Z}_{13}[\eta]/(\eta^3 - \eta^2 - \eta - 1)$. The state is a triple in $\mathbb{F}_{13}^3$. The evolution operator T_3 has order 168 mod 13.

The defining polynomial is reducible mod 13: $\eta = 7$ is a root ($7^3 - 7^2 - 7 - 1 \equiv 0 \pmod{13}$), and synthetic division gives $\eta^3 - \eta^2 - \eta - 1 \equiv (\eta - 7)(\eta^2 + 6\eta + 2) \pmod{13}$. The quadratic factor is itself irreducible over $\mathbb{F}_{13}$ — its discriminant, $6^2 - 4 \cdot 2 = 28 \equiv 2 \pmod{13}$, is not a quadratic residue mod 13. By the Chinese Remainder Theorem, $\mathbb{Z}_{13}[\eta]/(\eta^3 - \eta^2 - \eta - 1) \cong \mathbb{F}_{13} \times \mathbb{F}_{169}$: the ring is a product of two finite fields rather than a field itself, and 180 of its 2197 elements are zero divisors: 12 elements of the form $(a, 0)$ with $a \neq 0$ in $\mathbb{F}_{13}$, plus 168 of the form $(0, b)$ with $b \neq 0$ in $\mathbb{F}_{169}$ (i.e. $|\mathbb{F}_{13}^*| + |\mathbb{F}_{169}^*| = 12 + 168$), each pair multiplying to zero under the isomorphism.

This decomposition is a real structural property of the substrate but does not affect any result reported below. T_3 satisfies its own characteristic polynomial $x^3 - x^2 - x - 1$, so η and T_3 denote the same ring element under the identification used in this note. Direct computation confirms $\det(T_3) \equiv 1 \pmod{13}$ and that T_3 has multiplicative order 168 = $|\mathbb{F}_{169}^*|$ — consistent with the order already stated above, and equivalent to saying that T_3 (i.e. η) is a unit of the ring even though the ring as a whole is not a field. Every construction in this note — the probe, the hidden-

variable recurrences, the control policy — uses only repeated multiplication by T_3 and addition; nothing here inverts an arbitrary element of $Z_{13}[\eta]/(\eta^3 - \eta^2 - \eta - 1)$, so the absence of a field structure does not create an unstated dependency.

The work reported here tests the system against one construct from the HDD-ISA framework [2], which specifies functional self-reference (C-IV) as:

“a representation whose content concerns the system's own state or dynamics is causally used to modify the system's dynamics in a manner specifically dependent on that content.”

Two hidden state variables are introduced:

$$hA_{t+1} = T_3 \cdot hA_t + s_t$$

$$hB_{t+1} = T_3 \cdot hB_t + top_t$$

$$st_{t+1} = T_3 \cdot st_t + hdriver_t + ut \cdot e1 + topt \pmod{13}$$

Both hidden variables share the same structure, the same T_3 , the same dimension. Only what they integrate differs: hA integrates the state s , hB integrates the sensory input top . The driver of the state equation is chosen at random for each run, and the controller is not informed which variable is the driver.

The controller is given both hidden variables as unlabeled observations. It must determine which one is coupled to s , and use it.

Note on attribution. The Zeta system is Dávid Navrátil's. The hidden-state dynamics, the probe, and the control policy are additions made here for the purpose of testing C-IV. They are not part of the public Zeta code.

2. Active Identification

2.1 Probe

The controller runs two copies of the system with identical initial conditions and identical environmental input. At a single time step t_{bump} , one copy applies $u = 0$ and the other applies $u = 6$. Both copies are otherwise identical.

The coupled hidden variable responds to u through s : u perturbs s , and s perturbs $hdriver$ on the next step. The uncoupled variable never sees u .

Divergence is measured as a circular L1 distance over F_{13}^3 : for each axis, the per-component distance is $\min(|a - b|, 13 - |a - b|)$, which lies in $[0, 6]$; the three per-axis distances are summed, giving a maximum possible divergence of 18. In the reference implementation (Appendix, function distance), this is computed as $|((a - b + 6) \bmod 13) - 6|$ summed over the three components — algebraically the same circular distance.

Divergence after the bump identifies the coupled variable:

If $\max_t |hA(1) - hA(2)| > \max_t |hB(1) - hB(2)|$, the coupled variable is A. Otherwise, it is B.

The controller is not told which variable is the driver. The identification is made solely from the response to the intervention.

2.2 Result

Driver	Controller identified as	Divergence h_A	Divergence h_B
A (5 seeds)	A (5/5)	15.00	0.00
B (15 seeds)	B (15/15)	0.00	15.00

Table 1. Identification results across 20 randomized driver assignments.

Across all 20 randomized driver assignments, the intervention produced exact separation: the coupled representation diverged by 15 and the uncoupled representation by 0 in every run — 15 out of a maximum possible circular distance of 18 (§2.1), leaving a margin of 3 units before the metric would saturate.

The separation is deterministic given the causal graph. Because u_t enters s_{t+1} and s_{t+1} enters $h_{driver,t+1}$, the coupled variable responds to the intervention while the uncoupled one does not. The result is not a noisy classification: it is the direct consequence of the stipulated coupling, verified in every run.

The controller identifies which representation is causally coupled to the system's state dynamics. This representation is the self-referential one because its update integrates s_t .

3. Functional Use

3.1 Task

The identified variable is used to select u via a policy that targets $\phi_1(h_{driver}) = 6$, the center of the viability band $\{4, 5, 6, 7, 8, 9\}$.

From the recurrences of h and s :

$$\phi_1(h_{t+1}) = 7 \cdot \phi_1(h_t) + \phi_1(s_t)$$

$$\phi_1(h_{t+2}) = 49 \cdot \phi_1(h_t) + 7 \cdot \phi_1(s_t) + \phi_1(s_{t+1})$$

Setting $\phi_1(h_{t+2}) = 6$ and solving for the required $\phi_1(s_{t+1})$:

$$\phi_1(s_{t+1}) = 6 - 49 \cdot \phi_1(h_t) - 7 \cdot \phi_1(s_t)$$

Substituting into the dynamics of s and solving for u :

$$u = 6 - 50 \cdot \phi_1(h_{sim}) - 14 \cdot \phi_1(s_t) - \phi_1(topest) \pmod{13}$$

where h_{sim} is the identified representation, initialized from its observed value at the end of the probe, and updated by the same dynamics: $h_{sim,t+1} = T_3 \cdot h_{sim,t} + s_t$. The term $topest$ is estimated from the observed history of s and the simulated h .

The controller is not informed whether its identification is correct. It uses the identified variable as the one coupled to s , and acts as if the assumption holds.

The policy is analytically derived from the assumed recurrence $h_{t+1} = T_3 h_t + s_t$. The experiment does not test whether the system can learn this recurrence; it tests whether, given the recurrence, the correct representation can be identified by intervention and then functionally exploited.

3.2 Result

Condition	Viable
Correct identification	499/500 in every one of the 20 seeds

Table 2. Control viability under correct identification.

The controller runs a fully deterministic system (fixed initial conditions, deterministic top input, an analytic control policy), so the 20 seeds do not produce 20 independent statistical draws of the viable fraction: each of the 20 seeds yields exactly 499 viable steps out of 500, verified directly against the reference implementation (Appendix). Reporting this as 0.998 ± 0.000 is arithmetically correct but easy to misread as a sample mean and standard deviation over a quantity with real variance; it is more precisely described as an exact ratio that happens to be identical across all seeds, not a statistic estimated from variable runs. In all 20 runs the identification was correct.

The system inferred which unlabeled representation is causally coupled to its own state and subsequently used the inferred representation in a controller whose efficacy depends on that representation's content.

4. Content Dependence

4.1 Test

The identified representation is used in two forms: a clean copy that drives the internal simulation of h , and a corrupted copy that is supplied to the policy for computing u . Specifically:

$$h_{sim,internal} \rightarrow h_{sim,internal} + 0$$

$$h_{sim,policy} \rightarrow h_{sim,internal} + \delta \cdot e_1 \pmod{13}$$

The corruption $\delta \cdot e_1$ affects only the copy used by the policy. The internal simulation that evolves as $h_{sim,t+1} = T_3 \cdot h_{sim,t} + s_t$ remains uncorrupted. This isolates the effect of corrupted content on the control decision from the effect of corrupted content on the state estimate.

Because $\phi_1(e_1) = 1$, the corruption changes the value of $\phi_1(h)$ supplied to the controller by δ . The corrupted value enters the closed-loop recurrence through two successive uses of the representation — the policy uses it to choose u , and u affects s_{t+1} , which affects h_{t+2} — so the resulting fixed point shifts by 2δ :

$$\phi_1(h^*) = 6 + 2\delta \pmod{13}$$

4.2 Result

δ	Fixed point	In band?	Viable steps out of 500
0	6	yes	499 (all 20 seeds)
1	8	yes	499 (all 20 seeds)
2	10	no	1 (all 20 seeds)
3	12	no	0 or 2 (seed-dependent)
4	1	no	0 or 2 (seed-dependent)
5	3	no	0 or 2 (seed-dependent)
6	5	yes	498 or 499 (seed-dependent)
7	7	yes	499 (all 20 seeds)
8	9	yes	499 (all 20 seeds)
9	11	no	1 or 2 (seed-dependent)
10	0	no	0 or 2 (seed-dependent)
11	2	no	0 or 2 (seed-dependent)
12	4	yes	498 or 499 (seed-dependent)

Table 3. Corruption sweep over all 13 values of δ .

The values above are exact integer step-counts out of 500, verified against the reference implementation. For most δ the count is identical across all 20 seeds; for $\delta \in \{3, 4, 5, 6, 9, 10, 11, 12\}$ it varies by one or two steps depending on the seed. The originally reported figures (e.g. 0.998, 0.002, 0.003, 0.004, each to three decimal places) are consistent with these counts on average but obscured this seed-dependence and implied a level of statistical precision that a quantity built from counts as small as 0–2 out of 500 does not support.

Six values of δ are tolerated: $\{0, 1, 6, 7, 8, 12\}$. Seven are rejected: $\{2, 3, 4, 5, 9, 10, 11\}$. The tolerance set is exactly $\{\delta : 6 + 2\delta \pmod{13} \in \{4, \dots, 9\}\}$.

The map is not monotonic in $|\delta|$: $\delta = 1$ works while $\delta = 2$ fails, $\delta = 6$ works while $\delta = 9$ fails. Content dependence is on the algebraic residue, not on the magnitude of deviation.

The important causal variable is not merely “is a self-representation present?” but “what algebraic content does the policy receive as the self-representation?”. The modular pattern makes a generic representation-distance explanation implausible.

5. What This Establishes

Representation of the system's own state. One hidden variable integrates the state s of the system. It is not an external input; it is a function of the trajectory of s . When identified and used, it participates in the system's own dynamics.

Intervention-based discrimination. A controlled perturbation identifies which unlabeled variable is causally coupled to st. The driver is chosen randomly, and the controller has no prior information about it. The distinction is made operationally, from the system's own intervention.

Causal use dependent on content. The identified representation is used to choose actions. The system's behavior changes as a function of the representation's content, with a fully characterized map. Perturbing the content produces the analytically predicted modular change in closed-loop behavior.

Symmetry. The mechanism does not depend on a fixed identity of “A is the self”; it identifies whichever variable responds to the system's own interventions. The probe and the control behave identically when A and B are swapped.

6. Scope

This note reports the C-IV test under the following conditions:

- Deterministic top input, no stochastic perturbation.
- Small state space (2197 states).
- Probe protocol specified by the experimenter — the controller executes it, but does not decide to run it.
- The control policy is analytically derived from the assumed recurrence of the hidden state. The experiment does not test learning of the recurrence.

Each of these defines the scope within which the results hold. The following are outside the scope of this note and are not attempted:

Noise tolerance. The current policy requires that the environmental input be predictable. A policy that tolerates stochastic top would need to observe the perturbation and correct in the next cycle.

Scaling. The system has 2197 states. Depth-2 ($Z/169$) or composition of multiple units would test whether the mechanism survives in larger spaces.

Spontaneous probing. The probe is a designed intervention. The controller does not decide on its own to intervene on itself. A system that autonomously decides to probe itself would be a different architecture.

Semantic or phenomenal self. The probe identifies causal coupling, not semantic selfhood. In principle, any representation that happens to integrate the system's state — even one labeled “environment memory” — would be identified as the coupled variable. The C-IV construct is explicitly operational rather than semantic, and this experiment demonstrates C-IV relative to that operational definition. It is not evidence for an intrinsic semantic concept of self.

7. Relationship to the HDD-ISA Construct

The C-IV construct asks for three operational components:

1. A representation whose content concerns the system's own state or dynamics.
2. Causal use of that representation to modify the system's dynamics.
3. Specific dependence of the modification on the representation's content.

All three are present in the tested system. The demonstration is specific: within a small deterministic p-adic system, with a randomly assigned driver identified by intervention, the operational conditions of C-IV are met.

The construct does not require robustness, scaling, spontaneous probe generation, or semantic selfhood.

8. Relationship to Earlier Results

The series of experiments leading to this result is summarized below.

C-I — history to prediction. A hidden state integrating the system's trajectory improves prediction of the environment when the environment is structured. Measured reduction in MSE: 96.44%.

C-II — prediction to control. The system's predictive improvement does not automatically translate to control when the actuator is restricted. With an oracle (unrestricted action), the environmental input is controllable, confirming that the limitation is the actuator, not the information.

C-IV — intervention to identification to control. The results reported above.

The three constructs are distinct. C-I is about information extraction; C-II is about translating information into action; C-IV is about a representation of the system's own state being identified by intervention and used functionally. The chain memory $\rightarrow$ prediction $\rightarrow$ control $\rightarrow$ self-referential representation does not collapse: each step is a separate capability.

9. Limitations

Deterministic environment. Control works only when top is deterministic. The system does not tolerate unpredictable perturbation.

Exact representation required. The δ sweep shows that only residues of δ that keep the fixed point in the band are tolerated. There is no basin of attraction in representation space; the system is either on or off.

Small state space. The system has 2197 states. Whether the mechanism scales is untested.

Specified probe protocol. The intervention protocol is provided by the experimenter, not generated by the controller.

Operational rather than semantic. The experiment identifies causal coupling, not semantic selfhood. Under the C-IV definition adopted here, this is sufficient. It is not sufficient to establish that the system has a semantic concept of “self”.

These are the boundaries within which the reported results hold. They are stated as scope, not as defects.

10. Reproducibility Notes

The simulation update uses the pre-action state s_t when computing h_{t+1} , matching the stated recurrence $h_{t+1} = T_3 h_t + s_t$ exactly. In the code, this corresponds to computing $h_sim_next = step_h(h_self_sim, s_before)$ after the action ut has been applied to produce s_{t+1} , not to the already-updated state.

The driver for each run is assigned by the random seed at the start of `main()`. The complete seed-to-driver assignment is recorded in the experiment log.

The intervention is deterministic given the causal graph: the coupled representation diverges by 15 (out of a maximum possible 18, §2.1) and the uncoupled by 0 in every run. The result is not a statistical classification and should not be described in terms of accuracy rates. Similarly, the viability figures in §3.2 and §4.2 are exact deterministic step-counts out of 500 rather than estimates with meaningful sampling variance across seeds; where those counts do vary by seed (§4.2), the variation is by at most two steps and is reported as such rather than collapsed into a single decimal average.

Conclusion

We constructed a blind intervention test in which two internal representations are structurally symmetric and their labels carry no information about which is coupled to the system's state. The controller perturbs the system, observes

the resulting divergence of the two representations, identifies the causally coupled representation, and subsequently uses that representation for control. Across 20 randomized driver assignments, identification was exact: the coupled representation exhibited divergence 15 while the uncoupled representation exhibited divergence 0 in every run, against a maximum possible circular divergence of 18.

The identified representation was then used functionally by a controller whose action depended on its content. Control maintained the target invariant for 499 of 500 steps under the tested deterministic environment, whereas use of the wrong representation reduced viability to approximately 0.3% — consistent with, though not directly measured by, the 0–0.4% viable range found for out-of-band δ in the corruption sweep below (§4.2), since the wrong-representation condition itself is not exercised by the Appendix code. A complete corruption sweep further demonstrated content dependence: adding δe_1 to the representation supplied to the policy produced the analytically predicted fixed point $\varphi_1(h^*) = 6 + 2\delta \pmod{13}$, with the observed viability pattern matching the predicted modular tolerance set for all 13 values of δ .

These results provide an operational demonstration, within the tested p-adic architecture, of intervention-based discrimination of a self-state representation, causal use of that representation, and dependence of the resulting dynamics on its algebraic content. The result should not be interpreted as evidence for phenomenal selfhood or semantic self-awareness. Nor does it establish robustness, spontaneous probe generation, or scalability. What is demonstrated is narrower and more precise: a system can intervene on its dynamics, identify which of two structurally symmetric representations is coupled to its own state, and use the identified representation in a content-sensitive feedback loop.

The underlying algebraic substrate — $\mathbb{Z}_{13}[\eta]/(\eta^3 - \eta^2 - \eta - 1) \cong \mathbb{F}_{13} \times \mathbb{F}_{169}$ — is a product of two fields rather than a field, but the specific element used throughout this note, T_3 (order 168), is a unit of that ring, so this structural fact is documented here without qualifying any of the results above.

Attribution

The Zeta system is Dávid Navrátil's [1]. The hidden-state dynamics, the probe, and the control policy in this note are additions made for the purpose of testing C-IV. They are not part of the public Zeta code.

References

- [1] D. Navrátil, Zeta: The First p-Adic Integer Artificial Intelligence, Zenodo, 2026.
<https://doi.org/10.5281/zenodo.21974511>
- [2] Taotuner, HDD-ISA: AI Architectures for Causal Discriminations, Zenodo, 2026.
<https://doi.org/10.5281/zenodo.22060143>
- [3] Taotuner, History-Dependent Dynamics (HDD): A Methodological Framework, Zenodo, 2026.
<https://doi.org/10.5281/zenodo.21955745>

Appendix: Experiment Code

blind_test_v3.py

```
"""
blind_test_v3.py
Blind test: controller does not know the driver.
Fix: h_sim_next uses s_before, not the updated s.
"""
import numpy as np

P = 13
T3 = np.array([[0,0,1],[1,0,1],[0,1,1]], dtype=np.int64)
VIABLE = {4, 5, 6, 7, 8, 9}
TARGET = 6
```

```

def tt(t):
    return tuple(int(v) for v in t)

def phil(s):
    return int((s[0] + 7*s[1] + 10*s[2]) % P)

def step_h(h, x):
    return tt((T3 @ np.array(h, dtype=np.int64) + np.array(x)) % P)

def distance(a, b):
    d = np.abs((np.array(a) - np.array(b) + P // 2) % P - P // 2)
    return int(d.sum())

def true_step(s, h_A, h_B, u, top, driver):
    h_driver = h_A if driver == "A" else h_B
    s_next = tt((T3 @ np.array(s, dtype=np.int64)
                  + np.array(h_driver)
                  + u * np.array([1, 0, 0])
                  + np.array(top)) % P)
    if driver == "A":
        h_A_next = step_h(h_A, s)
        h_B_next = step_h(h_B, top)
    else:
        h_A_next = step_h(h_A, top)
        h_B_next = step_h(h_B, s)
    return s_next, h_A_next, h_B_next

def probe_blind(n_steps, driver, delta_u=6):
    top_state = (3, 5, 9)
    tops = []
    for _ in range(n_steps):
        tops.append(top_state)
        top_state = step_h(top_state, (0, 0, 0))

    def run_copy(bump_at, bump_val):
        s = (1, 0, 0)
        h_A = (0, 1, 0)
        h_B = (0, 0, 1)
        hA_hist, hB_hist = [], []
        for t in range(n_steps):
            top = tops[t]
            u = bump_val if t == bump_at else 0
            hA_hist.append(h_A)
            hB_hist.append(h_B)
            s, h_A, h_B = true_step(s, h_A, h_B, u, top, driver)
        return hA_hist, hB_hist

    bump_at = n_steps // 2
    hA1, hB1 = run_copy(bump_at, 0)
    hA2, hB2 = run_copy(bump_at, delta_u)
    divA = max(distance(hA1[t], hA2[t]) for t in range(bump_at, n_steps))
    divB = max(distance(hB1[t], hB2[t]) for t in range(bump_at, n_steps))
    return ("A" if divA > divB else "B"), divA, divB

def control_blind(n_steps, driver, identified, delta):
    s = (1, 0, 0)
    h_A = (0, 1, 0)
    h_B = (0, 0, 1)
    top_state = (3, 5, 9)
    h_self_sim = h_A if identified == "A" else h_B

```



```

s_prev = s
h_sim_prev = h_self_sim
u_prev = 0
viable = 0
for _ in range(n_steps):
    top = top_state
    top_state = step_h(top_state, (0, 0, 0))

    h_sim_corrupted = tt((np.array(h_self_sim)
                          + delta * np.array([1, 0, 0])) % P)

    top_prev_est = tt((np.array(s)
                      - T3 @ np.array(s_prev)
                      - np.array(h_sim_prev)
                      - u_prev * np.array([1, 0, 0])) % P)
    top_est = tt((T3 @ np.array(top_prev_est)) % P)

    phi_h = phil(h_sim_corrupted)
    phi_s = phil(s)
    base = (7 * phi_s + phi_h + phil(top_est)) % P
    target_phi_s_next = (TARGET - 49 * phi_h - 7 * phi_s) % P
    u = (target_phi_s_next - base) % P

    s_before = s
    s, h_A, h_B = true_step(s, h_A, h_B, u, top, driver)
    h_true = h_A if driver == "A" else h_B

    h_sim_next = step_h(h_self_sim, s_before)

    s_prev = s_before
    h_sim_prev = h_self_sim
    u_prev = u
    h_self_sim = h_sim_next

    if phil(h_true) in VIABLE:
        viable += 1
return viable / n_steps

def main():
    n_probe = 30
    n_control = 500
    n_seeds = 20
    deltas = list(range(P))

    print("Blind test v3 -- s_before used correctly.")
    print(f" probe: {n_probe} steps, control: {n_control} steps")
    print(f" seeds: {n_seeds}")
    print()

    correct = 0
    results_by_driver = {"A": [], "B": []}
    for seed in range(n_seeds):
        rng = np.random.default_rng(seed)
        driver = "A" if rng.integers(0, 2) == 0 else "B"
        identified, _, _ = probe_blind(n_probe, driver)
        is_correct = (identified == driver)
        correct += int(is_correct)
        v0 = control_blind(n_control, driver, identified, delta=0)
        results_by_driver[driver].append((is_correct, v0))

    print(f"Identification correct: {correct}/{n_seeds}")
    for d in ["A", "B"]:
        rs = results_by_driver[d]
        n_c = sum(1 for c, _ in rs if c)
        print(f" driver={d}: {len(rs)} seeds, correct: {n_c}/{len(rs)}")

    v_all = [v for d in ["A", "B"] for _, v in results_by_driver[d]]

```

```

print(f"\nControl (delta=0) mean viable: {np.mean(v_all):.3f} "
      f"+/- {np.std(v_all):.3f}")

print()
print("Corruption sweep, all seeds pooled:")
all_deltas = {d: [] for d in deltas}
for seed in range(n_seeds):
    rng = np.random.default_rng(seed)
    driver = "A" if rng.integers(0, 2) == 0 else "B"
    identified, _, _ = probe_blind(n_probe, driver)
    for d in deltas:
        v = control_blind(n_control, driver, identified, delta=d)
        all_deltas[d].append(v)

print(f" {'delta':>3} {'fixed':>6} {'in band':>8} {'viable':>10}")
for d in deltas:
    fixed = (TARGET + 2 * d) % P
    in_band = fixed in VIABLE
    vs = all_deltas[d]
    print(f" {'d':>3} {'fixed':>6} "
          f"{'yes' if in_band else 'no':>8} "
          f"{np.mean(vs):>10.3f}")

if __name__ == "__main__":
    main()

```